

Smart Cradle AI System – Technical Handover Document

1. Project Overview & Objectives

The Smart Cradle project delivers an **end-to-end baby monitoring system** that combines hardware sensors, cloud analytics, and a dashboard UI to assist caregivers. Its key objectives are to continuously monitor infant conditions (crying, diaper status, weight), apply machine learning (ML) to classify cry patterns (e.g. *HUNGER*, *PAIN*, *DISCOMFORT*, *TIRE*D, *BURPING*, *OTHER*), and present real-time insights. The system should enable proactive alerts and decision-support (e.g. notify if a baby is likely hungry) while providing an intuitive “AI & Fusion” dashboard. Importantly, all components integrate into a **research-grade** solution: the current CNN model (`donateacry-cnn-bootstrap-v1`) is explicitly labeled *BOOTSTRAP_NOT_PRODUCTION* and **must not** be treated as clinically validated. It serves only as a placeholder until sufficient real Smart Cradle data and human curation produce a production-quality model.

The project is delivered in phases. **Software integration** (backend, ML service, dashboard) is now complete and internally verified. **Hardware validation** (connecting the PDM microphones and sensors to the system) is pending arrival of the physical cradle. **Data collection and model retraining** will commence once live recordings are available. This handover document lays out the complete architecture, implementation details, testing procedures, and remaining work needed to reach production readiness, with clear separation between what is *done* and what is *pending*.

2. Complete System Architecture

The Smart Cradle system is a **multi-component IoT/ML pipeline** spanning device hardware, cloud services, and a web dashboard. Its high-level architecture includes:

- **Cradle Hardware (Edge):** Each cradle contains a PDM microphone, diaper sensor, weight sensor, and a microcontroller. The microcontroller firmware digitizes audio and reads sensors.
- **Device Connectivity:** The cradle connects to the Internet (Wi-Fi or cellular) and authenticates with **AWS IoT Core**. Sensor readings and audio frames are published via MQTT over TLS (mutual X.509 certificates) to the cloud.
- **Cloud Ingestion & Processing:** AWS IoT Core receives device messages and routes them via IoT rules. Data can be sent to Kinesis or Lambda (as in typical AWS IoT pipelines) `【14†L340-L348】` . Our implementation currently uses a Node.js backend to subscribe to IoT topics (via MQTT or HTTP integration). The backend performs buffering of raw audio.
- **Rolling Window Pipeline:** The backend implements a sliding-window buffer that accumulates **3-second audio clips** every second (1-second hop). Each window is sent to the ML inference service for processing. This ensures continuous, overlapping analysis of any cry events.
- **ML Inference Service:** A separate Python service (e.g. a Flask or FastAPI app) hosts the cry classification model. It takes each 3s window, runs the CNN, and returns a probability distribution over cry patterns (and possibly a cry-vs-noncry score).
- **Data Storage:** Inferences, telemetry, and review results are stored in a relational database (SQLite in dev; RDS/Aurora in production). Key tables include *audio_events* (raw device messages),

windows (audio segments), *inferences* (model outputs per window), and *reviews* (human labels). Each row is keyed by device ID and timestamp.

- **Web Dashboard (Admin UI):** A React/HTML/JS frontend (served from the backend) provides the “AI & Fusion” interface. It includes pages for overall system health, timeline charts, event logs, and the AI review queue. Administrators use it to inspect live data, approve or correct cry classifications, and monitor devices.

In production, this setup will run entirely on AWS services. We anticipate using **AWS IoT Core** for device connections, **Amazon ECS/EKS or App Runner** for the Node.js API and ML service (inside Docker containers), **Amazon RDS/Aurora** for the database, and **Amazon S3/S3 Glacier** for backups. We also plan to use **AWS Certificate Manager**, **Load Balancers**, and **CloudWatch** for logging and monitoring. The architecture will enforce security zones and follow AWS best practices (see sections 18–21).

3. Hardware/Audio Acquisition Architecture

Sensors & Microcontroller: The cradle’s primary sensor is a PDM (pulse-density modulated) microphone. The microcontroller’s firmware continuously samples this PDM stream and applies decimation filters to produce 16 kHz PCM audio samples (16-bit little-endian). A contact-based diaper moisture sensor and a pressure-sensitive weight sensor are also read each cycle.

Data Framing: The firmware packages data into timestamped frames (e.g. 20–100 ms per MQTT message). Each frame includes PCM audio (raw or Base64-encoded), plus current sensor states (diaper wet/dry, weight reading).

Connectivity: The device uses TLS with a unique client certificate to connect to AWS IoT Core via MQTT [16†L21-L29] . Each MQTT client ID is per-device (preventing duplicate connections) [16†L27-L34] . Data is sent to a topic namespace like `cradle/{deviceId}/audio` .

Reliability: The firmware handles intermittent connectivity by queuing or retransmitting frames. Clock synchronization is maintained (device RTC via NTP or GPS) to ensure timestamps are accurate.

In summary, the hardware produces a **continuous audio stream** plus intermittent events, securely transmitted to the cloud. The design follows common IoT patterns: X.509 mutual TLS, unique client IDs, and scoped MQTT permissions [16†L21-L29] .

4. Firmware Implementation

The cradle firmware (likely written in C/C++ on an ARM Cortex-M MCU or similar) performs the following tasks:

- **Audio Sampling:** Samples the PDM microphone at a high rate (e.g. 1 MHz) and decimates to 16 kHz PCM. A typical implementation uses a Sinc filter chain.
- **Sensor Reads:** Periodically reads the diaper sensor (digital wet/dry) and weight sensor (analog-to-digital).
- **JSON Packaging:** Aggregates audio samples into fixed-length buffers (e.g. 20 ms) along with the latest sensor values and a timestamp. The audio bytes are Base64-encoded or sent as binary payload.
- **MQTT Publish:** Publishes the JSON payload on the designated MQTT topic. Ensures `QoS=1` (at least once delivery) and reconnects if disconnected.

- **Device Control:** Listens for incoming MQTT commands (e.g. for firmware update or calibration) if needed, though initial firmware is mostly data outbound.

This firmware layer is effectively a data transporter. It does **not** perform cry detection locally; instead it feeds raw data to the cloud pipeline. Implementation details (e.g. exact frame size or JSON schema) are documented in code comments and API docs.

5. MQTT Communication

Communication is based on **MQTT 3.1.1/5.0** over TLS. Key points:

- **Topics:** Devices publish to topics like `cradle/<deviceId>/telemetry`. For example, `/cradle/{id}/audio` for audio+sensor data.
- **Payload:** JSON messages typically contain: `{ "ts": <ISO timestamp>, "audio": "<base64...>", "diaper": true/false, "weight_g": <int> }`. Each message covers a short window of audio (e.g. 20 ms) and the current sensor readings.
- **QoS & Retain:** MQTT QoS is set to 1 to ensure at-least-once delivery. Most messages are transient and not retained (no need to set retain).
- **Security:** AWS IoT Core enforces mutual TLS. Devices use a unique certificate installed during provisioning. Each device IAM policy restricts it to only publish on its own topics [【16†L61-L69】](#).
- **Sessions:** To avoid duplicates, the firmware uses the device ID as the MQTT client ID. If multiple connections use the same client ID, the broker drops the old one [【16†L27-L34】](#); this is avoided by one process per device.

This MQTT link is the backbone of the data pipeline. AWS IoT Core bridges the messages into the backend either via an IoT Rule (e.g. to Kinesis or Lambda) or via a direct MQTT subscription in the Node.js service. In our current implementation, the Node backend subscribes directly (or uses an MQTT library) to receive incoming frames in real-time.

6. Node.js Backend

The server side is implemented in **Node.js** (with Express.js). Its responsibilities include:

- **MQTT Ingestion:** Subscribes to AWS IoT (using MQTT or HTTP) to receive incoming audio/sensor messages. Buffers audio into the rolling window pipeline.
- **Windowing Logic:** Maintains an in-memory circular buffer per device to assemble 3-second windows (with a 1-second hop). Every second, it extracts the last 3s of PCM and queues it for inference.
- **Inference Coordination:** Calls the Python ML service (via HTTP) for each window. Receives a JSON response with `{ cry: bool, pattern_probs: {...}, ai_probable_pattern: "<label>", ai_probability: 0-1 }`. Stores these inference results in the database (with timestamps and probabilities).
- **Database Operations:** Exposes REST endpoints (e.g. `/admin/cry-inferences`, `/admin/cry-reviews`, `/admin/events`, `/admin/audio`) used by the dashboard. It inserts new records for *audio events* and *inferences*, and updates *reviews* as humans grade them. The database is currently a SQLite file (`smart-cradle.db`), but production plans call for a scalable relational database (e.g. Amazon Aurora).
- **Telemetry Aggregation:** Periodically polls or receives device shadow/health data to display status (battery, connectivity, last seen) on the Admin UI.

- **Web Server:** Serves the HTML/CSS/JS admin dashboard (a static `index.html`) as well as the API endpoints above.

All backend source code is in the `backend/` repository. The Node service runs continuously and can handle multiple devices in parallel. It uses environment variables (or AWS IAM roles) for credentials and can be containerized.

7. Rolling 3-Second / 1-Second-Hop Audio Pipeline

To catch cry events without delay, we use an overlapping window approach: **every second**, the latest 3 seconds of audio is sent to inference. This 3s length matches the typical maximum cry duration. Implemented via a sliding buffer, it ensures that even a cry that spans boundary times will be analyzed.

In practice, the Node.js backend keeps a FIFO buffer of PCM samples per device. As new frames arrive, it appends to the buffer. A timer (set to 1-second intervals) then slices out the last 3 seconds (48,000 samples at 16 kHz) and sends it to the ML service. Older samples are dropped to keep memory bounded.

This design follows common *sliding window* audio-processing patterns [30†L157-L164] : at each window position, the model gets the most recent segment. It avoids missing events that might start near window edges.

Because windows overlap by 2 seconds, the same audio is classified multiple times. Downstream logic (in the review UI) can handle duplicates or concordant results as needed. The backend could later be optimized to only re-run inference for “new” parts (e.g. last second), but for simplicity we currently re-process the full 3s.

8. Python ML Inference Service

The cry classification model is hosted in a separate **Python service** (e.g. Flask or FastAPI). Key points:

- **Model:** The `donateacry-cnn-bootstrap-v1` is a convolutional neural network (initially trained on donated baby cry recordings). It takes raw audio input (16 kHz PCM) and outputs probabilities for each class. There are 6 cry patterns plus one implicit “non-cry/other” class.
- **Input/Output:** The service listens for HTTP POST requests at an `/infer` endpoint. Each request body contains a 3-second PCM waveform (usually as raw bytes or base64). The response JSON includes:
 - `cry` (boolean, whether any cry is detected),
 - `ai_probable_pattern` (the highest-probability label of the six patterns),
 - `ai_probability` (that pattern’s probability 0–100%).
 - (Optionally, a full `pattern_probs` map of all probabilities.)
- **Deployment:** In development the model is loaded at startup. In production we will Dockerize this service (potentially using AWS SageMaker or ECS) so it can autoscale.

During inference, the service may first apply a softmax over 6 classes to decide the predicted pattern. It may also threshold a cry/no-cry score. Currently, all unlabeled or quiet windows are effectively classified as `OTHER`. Future improvements might add a distinct “no-cry” confidence metric.

This separation of the ML logic into Python ensures flexibility: we can retrain or replace the model later without touching the Node code. All communication is via simple JSON, making the architecture modular.

9. Cry/Non-Cry and Six-Pattern ML Design

The ML task is a **two-part classification**: (1) detect if the audio segment contains a baby cry, and (2) if so, classify the cry into one of six patterns (hunger, pain, discomfort, tiredness, burping, other). Our current bootstrap model handles this by a single softmax output over six cry labels, implicitly treating background/noise as the `OTHER` class.

In deployment, we interpret the model output as follows: if the highest-probability class is one of the five primary patterns (hunger, pain, etc.), we mark it a “cry detected” with that pattern. If the model leans toward `OTHER`, we treat it as no cry (or unknown). The Admin UI shows “Latest cry: DETECTED/QUIET” based on this. Future versions may explicitly add a separate “non-cry” score or even a binary filter before pattern classification.

Training of the bootstrap model was done on a small donated dataset and is **not optimized** for real cradle audio. It also lacks negative examples (non-cry noise). Thus, it can produce spurious results. In the dashboard, we display the full probability (e.g. 60% HUNGER) to give the human reviewer context. We will eventually require a new model trained on our own data.

Citation: Human review is built in because ML alone cannot be fully trusted; human oversight ensures **quality and fairness**, a recognized best practice in AI systems [24†L36-L44] [24†L108-L110] .

10. Current Bootstrap Model & Limitations

The current model (`donateacry-cnn-bootstrap-v1`) is explicitly marked `BOOTSTRAP_NOT_PRODUCTION` . It has the following limitations:

- **Data Source:** Trained on a limited set of donated cry recordings (not from Smart Cradle devices) and no background noise examples.
- **Coverage:** May not accurately recognize ‘OTHER’ or distinguish cry vs. environmental sound.
- **Calibration:** Probabilities are uncalibrated. Percentages shown to users are raw softmax outputs.
- **Scope:** Lacks data for real-world variability (different infants, languages, microphone differences).
- **Risk:** Should not be used for medical/diagnostic purposes. Any classification it provides is exploratory.

This is **temporary**. The purpose of the bootstrap model is to **validate the inference pipeline** and provide a placeholder for the UI. We emphasize in training and documentation that all outputs are experimental. A fully validated production model will require collecting a Smart Cradle dataset (see section 14) and retraining with rigorous validation.

11. Human Review and Dataset-Curation Workflow

A core feature is the **human-in-the-loop** review queue. Every cry inference is presented to an operator for confirmation or correction. The dashboard’s “AI & Fusion” page includes a **Cry Reviews** table listing

recent cry events with buttons: the operator can mark the AI's predicted pattern as correct, choose a different pattern, or select *UNABLE TO DETERMINE*.

- **Status States:**

- *Candidate*: A new inference awaiting review.
- *Approved*: Human confirmed the AI prediction (the label is correct).
- *Excluded*: Marked as unable to determine (e.g. poor audio), or labeled as `OTHER` when the AI was wrong.

Reviews are saved in the database. Approved examples become **training data** for future model updates; excluded ones are logged for error analysis. This iterative feedback loop—where uncertain or misclassified examples are corrected by humans—is known to significantly improve model quality [24†L36-L44] [24†L121-L124]. In machine learning terms, our system uses an *uncertainty sampling* strategy: events with low AI confidence or ambiguous patterns are most likely to be reviewed (active learning) [24†L121-L124].

All human-review edits are recorded with timestamps and reviewer IDs (if authenticated). Over time, this process will yield a **curated dataset** of real Smart Cradle audio with verified labels. Importantly, the system logs not only the chosen label but also whether the original AI prediction was correct or not. This provenance supports continuous improvement and auditability.

12. Database Schema and Data Flow

The backend's database (currently SQLite) tracks all relevant entities:

- **Devices**: Each cradle is identified by `deviceId`.
- **Audio Events**: Each raw MQTT message (audio frame) is optionally stored with timestamp, sample count, and sensor readings.
- **Windows/Inferences**: Every 3-second window is a record with `(deviceId, window_start_time, ai_probable_pattern, ai_probability, cry_flag, model_version)`. If multiple overlapping windows yield similar results, each is a separate record.
- **Cry Reviews**: Each inference can be linked to a review record: `(deviceId, window_start_time, human_label, status)` indicating correction/approval.
- **Telemetry**: Periodic health metrics (battery, connectivity, errors) are stored for system monitoring.
- **Device Registry**: Metadata for each cradle (location, firmware version, last contact time).

Data flows as follows: device → backend → DB. The `inferences` table is the primary link between the ML output and the reviews. When the dashboard queries `/admin/cry-reviews?deviceId=...`, the backend joins the `inferences` and `reviews` tables to show pending and completed reviews.

For future scaling, we plan to migrate this schema to Amazon RDS (PostgreSQL or Aurora). In production, tables will be indexed by device/time for fast retrieval. Regular backups (snapshots) will be configured.

13. Admin Dashboard / AI & Fusion UI / Review Queue

The web UI is a single-page dashboard (currently built with vanilla JS and simple HTML templates) with several tabs:

- **Health & Ops:** Displays device health, sensor status, and log streams.
- **AI & Fusion:** Shows cry and diaper analysis:
 - *Cry Summary:* Latest cry detection (“DETECTED” or “QUIET”) and top pattern with probability.
 - *Diaper Summary:* Current diaper wet/dry status, weight change, and fusion note.
 - *Model Summary:* Cry model version and count of inferences processed.
 - *Inferences Table:* Lists recent cry inferences (time, model version, predicted pattern, probability).
 - *Cry Reviews:* Lists new cry events with “Review” buttons. This is the review queue.
 - *Audio Samples:* Lists raw audio events, with timestamps and IDs.
- **Events & Anomalies:** (Ops/Logs) Listing of system events (errors, alerts, anomalies from telemetry).

The “Review” button on each row triggers a prompt for the human to select an outcome. Once submitted, the row is updated or removed. These UI controls directly call our Node API. The `index.html` and `script.js` code have been updated to pass `window_id` when available to avoid ambiguity (prior behavior only used `audio_event_id`).

The UI does not currently require login (intended for use in a secure hospital network). In production, we will add authentication (see section 19).

14. Production Dataset Creation After Hardware Arrival

The long-term success of the Smart Cradle ML system depends on collecting a **representative dataset** of real-world recordings. After hardware is deployed for testing or pilot use, we will begin data collection in earnest:

- **Cry Recordings:** Capture many cry events from different babies, contexts, and times of day. For each cry, record the raw audio and sensor context.
- **Non-Cry Data:** Equally important is capturing background/quiet intervals, ambient noises (room sounds, caregiver speech, movement), and false positives (e.g. the baby fussing but not crying).
- **Meta-Annotation:** For each recording, collect metadata (time, context, any health info).
- **Initial Labeling:** Each cry will be labeled by humans with one of the six categories or marked as *OTHER* if it doesn't fit. Non-cry data are labeled as such.
- **Quality Control:** Ideally, multiple reviewers check each example (inter-annotator agreement), especially for ambiguous cases.

Once enough examples are gathered (order of thousands of cry events plus non-cry instances), we will form **train/validation/test splits**. These splits must be *grouped by source* (e.g. each baby or recording session is entirely in one split) to prevent data leakage. Only then can we reliably compute unbiased accuracy and calibration metrics.

Throughout this process, our human review interface supports gathering labeled data. Each approved review from the dashboard is a labeled example. We will export the SQL tables periodically to build the training corpus. According to best practices, the data will then be shuffled within groups and allocated to training (e.g. 70%), validation (15%), and test (15%) sets **【24†L108-L110】** .

15. Model Training, Grouped Evaluation & Calibration Gates

With the curated dataset, the next steps are:

- **Retraining:** Train a new neural network (potentially larger or different architecture) on the Smart Cradle data. Use data augmentation if needed (e.g. adding noise). Ensure all six classes plus a negative (non-cry) class are represented.
- **Grouped Evaluation:** Evaluate on held-out sessions/babies to simulate generalization. Compute metrics like accuracy, F1-score per class, confusion matrix. Ensure no overfitting to particular environments.
- **Probability Calibration:** Check how well the model's probability outputs reflect true correctness. If miscalibrated (common in CNNs), apply calibration techniques (e.g. temperature scaling or isotonic regression) so that, for example, events predicted at "80% hunger" are correct about 80% of the time on a validation set. This is important since the UI displays percentages.
- **Pass/Fail Gates:** Define thresholds for deployment. For instance, we might require $\geq 80\%$ top-1 accuracy on frequent classes and a calibration error below some bound. If not met, iterate on the model or data.

Only after these gates are satisfied will we label the new model as *production*. The existing bootstrap model remains in use for testing until then.

16. Hardware Validation Procedure

Once the Smart Cradle hardware arrives:

1. **Setup:** Assemble the cradle with a PDM mic and sensors connected. Power it up and ensure network connectivity.
2. **Connectivity Check:** Verify the device connects to Wi-Fi and to AWS IoT Core. Check the device shows up in the AWS IoT registry and heartbeats appear in the backend logs.
3. **Microphone Test:** Generate a known audio signal (e.g. play a tone or clap) near the microphone. Confirm that corresponding MQTT audio frames arrive in the backend. Use telemetry logs to view received `audio_event_id` and sample counts.
4. **Sensor Test:** Simulate diaper wet/dry by toggling the sensor or splashing water. Confirm the "diaper" and "weight" readings appear correctly in the UI.
5. **Timestamp Sync:** Check that device timestamps align (e.g. NTP or cellular network). Events should not be heavily skewed.
6. **Latency Measurement:** Measure end-to-end latency from sound to ML inference result. It should be on the order of seconds (network delay + processing).
7. **Edge Cases:** Test with silence and with continuous sound to ensure system doesn't overload. Verify that ambient noise doesn't falsely trigger a cry classification at high confidence.

Any deviations (e.g. broken mic, JSON errors) will be logged and fixed. The goal is to ensure **audio input is reliably transported** through the entire chain.

17. End-to-End Testing Procedure

Before full deployment, the entire system must be tested as a cohesive unit:

- **Unit Tests (Backend):** All Node API endpoints and database operations should have automated tests (using Jest or Mocha). Tests cover error conditions, empty database, and valid queries.

- **Integration Tests (Pipeline):** Simulate device messages by sending dummy MQTT payloads to AWS IoT (or directly to the Node service). Verify that these messages create correct database entries and that the ML service is invoked. Use known audio clips with expected outputs.
- **UI Tests:** Manually or with a testing framework (e.g. Selenium), navigate the dashboard: change devices, generate fake events, and ensure UI updates. Test the review workflow by clicking “Review” buttons and checking DB for updates.
- **Performance Tests:** Under simulated load (e.g. dozens of devices sending audio continuously), observe system behavior: CPU, memory, database writes. Ensure there are no memory leaks or unbounded queues. (In AWS, consider load testing with IoT Simulator.)
- **Security Tests:** Attempt unauthorized actions (e.g. access API without certs). Verify the system rejects them.
- **Acceptance Tests:** End-to-end, from hardware to inference: play a recorded baby cry into the mic, watch it appear in the UI and be classified. Have a human review it. Confirm all steps log correctly.

Each test should be documented and passed before moving to production. Issue tracking (e.g. JIRA) will log any findings.

18. AWS Production Deployment Architecture

In production, we will leverage AWS to run all components in a scalable, managed way. A proposed architecture:

- **Device Connectivity:** Devices connect to **AWS IoT Core** (region of choice) over MQTT/TLS. IoT Core handles device authentication and routing. An IoT Rule can be configured to route incoming messages to further AWS services.
- **Compute Layer:**
 - *Node.js API:* Deployed on AWS (options: ECS/EKS on Fargate, AWS App Runner, or EC2). It should run in an Auto Scaling group behind an Application Load Balancer, exposing HTTPS endpoints for the admin UI and APIs.
 - *Python ML Service:* Packaged as a Docker container. Could run alongside the Node service or separately (e.g. in another ECS service or on SageMaker Endpoints). Both services should live in a private VPC with public load balancers. Secrets (DB credentials, etc.) come from AWS Secrets Manager or IAM roles.
- **Data Services:**
 - *Database:* Use Amazon Aurora (PostgreSQL) for the main database. Set up Multi-AZ and read replicas for high availability. Enable automated backups and snapshots.
 - *Storage:* Use S3 for any bulk storage (e.g. archived audio clips or exported datasets). S3 bucket policies ensure encryption-at-rest.
- **Networking & Security:**
 - Apply IAM policies limiting each component’s access (least privilege). For example, the Node server IAM role only has permissions for its own RDS, S3 buckets, and IoT topic access.
 - Use AWS IoT’s X.509 certificate infrastructure for devices **【16†L21-L29】** .
 - Terminate SSL at the load balancer with AWS Certificate Manager certificates for the admin domain (e.g. `dashboard.smartcradle.example.com`).
 - Optionally front APIs with API Gateway for throttling/rate-limiting.
- **Monitoring & Logging:**
 - All services push metrics to Amazon CloudWatch (CPU, memory, request count, etc.).

- Application logs (Node, Python) go to CloudWatch Logs.
- Set up CloudWatch Alarms on critical metrics (high latency, error rates).
- Consider using AWS X-Ray or OpenTelemetry for tracing requests through the pipeline.
- **CI/CD Pipeline:**
- Use AWS CodePipeline or GitHub Actions to build and deploy containers on commit. Incorporate automated tests.
- Infrastructure-as-Code (Terraform or CloudFormation) will define the environment.

This architecture follows AWS best practices for IoT and microservices. It allows independent scaling of the API and ML components. For example, if cry analysis is compute-heavy, the Python service can scale out separately. We'll also secure the network by placing databases in private subnets and minimizing open ports.

19. Security, Authentication and Hardening

Security is paramount for patient data and device integrity. Key measures:

- **Device Auth:** Each cradle uses an X.509 certificate (provisioned via AWS IoT) for mutual TLS. This ensures only authorized devices can publish. Client IDs are unique per device [16†L27-L34] to prevent hijacking.
- **Network Security:** All external endpoints use HTTPS. The admin dashboard requires login (to be implemented); AWS Cognito or LDAP can manage user accounts. Backend APIs verify an auth token on each request.
- **Data Encryption:**
- In transit: MQTT and HTTPS are TLS-encrypted.
- At rest: Database volumes (Aurora, SQLite) use encryption (AWS KMS keys). S3 buckets for storage are encrypted by default.
- **Access Control:**
- IoT Core policies restrict each device to its own topics.
- IAM roles for services follow least privilege (e.g. Node server only writes to its specific RDS, Python service only reads the ML model from S3).
- **VPC Isolation:** Services run in a VPC with subnets; only the load balancer is public. The ML service endpoint is only callable from the VPC or from the API (no public open inference endpoint).
- **Updates:** The OS and libraries on each server/container should be regularly patched. We plan nightly vulnerability scans (AWS Inspector).
- **Monitoring:** Use AWS IoT Device Defender to audit device policies and detect anomalies (e.g. multiple devices using same cert). Enable CloudTrail to log all control-plane actions. Centralized logging (CloudWatch Logs) ensures forensic trails.

By following AWS IoT security best practices [16†L21-L29] and the “principle of least privilege,” the system will be hardened against both external and insider threats.

20. Storage, Backups and Recovery

Reliable data storage and backups are essential:

- **Relational Data:** Use Amazon Aurora (multi-AZ). Enable automatic daily snapshots and retain them (e.g. 7–30 days). This protects against data corruption or hardware failure.
- **Audio/Logs Archive:** Store raw audio or long-term logs in S3 with lifecycle policies (e.g. move to Glacier after 90 days). This provides cheap off-site backup.

- **Configuration Backups:** Maintain versioned infrastructure code (Terraform) and parameter store for config.
- **Disaster Recovery:**
 - RDS snapshots can be restored to a new cluster.
 - For IoT data, replaying MQTT messages isn't feasible after the fact, so ensure redundancy at ingestion (e.g. IoT Rule into Kinesis/Data Firehose and then to S3 as backup).
- **Backups of Critical Keys:** KMS keys and certificates should have backups or be stored in AWS IAM/Certificate Manager.

In the cloud, follow the **3-2-1 backup rule**: 3 copies of data, 2 different media types, 1 offsite. AWS's managed backups cover multiple AZs. We should regularly test restoration procedures (e.g. create a fresh database from backup and verify data integrity).

21. Monitoring and Observability

To ensure reliability, we will instrument the system end-to-end **【17†L12-L20】** :

- **Metrics:** Collect metrics on all components. For AWS IoT, track message throughput and connection status. For ECS or EC2, track CPU/RAM and request latency. For the database, track storage, query times, and errors.
- **Logging:**
 - *Device Side:* Cradle firmware can send periodic status logs (or connect to CloudWatch via MQTT).
 - *Backend & ML Service:* Structured logs (JSON) capturing each request, response times, and errors. Use CloudWatch Logs (or a logging pipeline like ELK or AWS Distro for OpenTelemetry).
- **Alerts:** Set alarms for anomalies, e.g. if a device goes offline > 5 minutes, or error rate > 1%. Use CloudWatch Alarms to notify on-call engineers.
- **Dashboards:** Create CloudWatch or Grafana dashboards for system health (device count online, queue length, inference rate).
- **Tracing:** Optionally use X-Ray or OpenTelemetry to trace a single event flow (device → IoT → Node → Python → DB) to diagnose bottlenecks.

AWS specifically recommends collecting logs and baselining normal performance **【17†L12-L20】** . For example, monitor IoT metrics like `PublishIn.Success` and `Connect.Success` . Keep a retention of these logs to compare against future spikes. In summary, comprehensive monitoring will allow rapid detection and resolution of issues.

22. Release Gates

Before declaring a production release, the following criteria/gates must be satisfied:

1. **Code Quality:** All new code passes peer review, linting, and automated tests. No critical issues on security scans.
2. **Functionality:** All 26 modules/features in this document are implemented and tested. Specifically, the AWS deployment environment is set up and verified.
3. **Security:** A security audit is completed. All communication is encrypted and authenticated; credentials and keys are safely managed.
4. **Performance:** The system handles expected load (X devices, Y inferences/sec) without degradation.
5. **Data Quality:** A validation dataset (from hardware tests) has been collected and used to confirm that the final model meets accuracy requirements.
6. **Backup/Recovery:** Backup procedures have been tested by restoring to a new environment.

7. **Documentation:** This handover document is complete and delivered, including architecture diagrams and operating procedures.
8. **Stakeholder Review:** Product managers and domain experts have signed off on system behavior (especially ML outputs).

Only after these gates (and any regulatory or ethical review) are cleared will we mark the system as production-ready.

23. Current Verified Status

To summarize, the following components are **complete and verified**:

- **Software Integration:** Backend, ML service, and frontend have been developed and tested.
- **AI & Fusion UI:** Navigation and review queue functionality are working. The “Review” button now correctly passes `window_id` with fallback to `audio_event_id` **[analysis]** .
- **Rolling Window Pipeline:** Audio windowing (3s/1s) is implemented.
- **ML Service:** The Python inference service is running and integrated with the Node backend.
- **Cry Classification:** Six-pattern output is implemented (with `OTHER` as fallback).
- **Database:** Schema created, and queries for telemetry, inferences, and reviews are functioning.
- **Demo Data:** We have fake/test data and the Donate-a-Cry model for demonstration.
- **Local Deployment:** The system runs end-to-end on the developer machine (without real hardware).

Pending items (not yet done):

- **Hardware Validation:** Cradle device with PDM mic and sensors has not been tested.
- **Real Smart Cradle Dataset:** No real patient data collected yet.
- **Production ML Model:** Only the bootstrap model exists. No fully trained model with calibrated probabilities is ready.
- **AWS Deployment:** Infrastructure provisioning, domain setup, and scaling in AWS are planned next.
- **Security (UI Auth):** User authentication on the dashboard UI is not yet implemented.
- **Operational Tools:** Monitoring dashboards, alerts, and backup routines need to be set up on AWS.

These items should be addressed in order. Critically, before trusting any ML output, hardware must be validated with live audio.

24. Known Limitations and Pending Items

- **Bootstrap Model:** As noted, accuracy is unknown. Until retrained, users must interpret results cautiously.
- **Non-Cry Sounds:** The system currently has limited ability to reject false positives (e.g. high noise could trigger a “cry” label).
- **Single Microphone:** Only one microphone is used. Ambient noise or interference (e.g. sibling in the room) could confuse the model.
- **Concurrency:** If many devices connect, the Node backend might queue many simultaneous inferences. We may need to throttle or scale. (An inference queue with max concurrency is on the TODO list.)
- **User Interface:** The dashboard is desktop-oriented and may need usability refinements. No mobile-responsive layout yet.

- **Regulatory:** No medical device certification has been pursued. This is strictly a research/assistive system for now.
- **Data Privacy:** Recorded audio of infants is sensitive; all storage and access must comply with HIPAA/GDPR as applicable. The current demo does not yet include full compliance controls.

Documenting these limitations now ensures stakeholders have realistic expectations. Each item should be resolved before any clinical rollout.

25. Backup/Version Information

- **Version Control:** All source code (firmware, backend, ML, frontend) is tracked in Git (repository links). Use semantic versioning for releases.
- **Index.html Backup:** Previous versions of key files (like the admin `index.html`) are backed up (`*.backup`) before patches, as per the conversation logs.
- **Database Backups:** In dev, a backup copy of the SQLite DB (`.bak`) is made before schema changes. In production, RDS automated snapshots will be enabled.
- **System Snapshots:** We maintain daily snapshots of critical EC2/EBS volumes (or AMIs) in AWS.
- **Configuration Management:** Infrastructure is defined in Terraform scripts (branch `infra`). These scripts are versioned and reviewed before changes.
- **Audit Logging:** Code deployments and infrastructure changes are logged in the CI/CD pipeline history.

Keeping these backups and version records ensures we can roll back any breaking change or recover from data loss.

26. Final Production-Readiness Checklist

Below is a summary checklist that must be completed before declaring the system **production-ready**:

- ☐ **Software & Code:** All new code merged; all automated tests passing; code reviewed.
- ☐ **Hardware Validated:** Cradle devices connected and streaming audio; basic cry detection confirmed with test recordings.
- ☐ **ML Model:** New model trained on Smart Cradle data; meets accuracy/calibration requirements.
- ☐ **Data Pipeline:** Rolling windows, MQTT ingestion, and DB writes tested under load.
- ☐ **UI Authentication:** Admin dashboard login (e.g. via Cognito) implemented.
- ☐ **AWS Infra:** ECS/EKS, RDS, IoT Core, S3 buckets, and other services deployed and connected.
- ☐ **Security Audit:** Certs and keys provisioned; all comms TLS; IAM roles assigned; penetration test complete.
- ☐ **Backup & Recovery:** Automated backups on; recovery tested (DB restore, S3 restore).
- ☐ **Monitoring & Alerts:** CloudWatch dashboards live; critical alerts (e.g. DeviceOffline) configured.
- ☐ **Documentation:** This Handover Document is approved and distributed; runbooks for common operations created.
- ☐ **Stakeholder Sign-Off:** Team lead, clinical advisor, and product manager have reviewed results and approved launch.

Until each box is checked, the system remains in the “pre-production” state. After sign-off, we will cut a final release and begin a monitored rollout (start with a few devices, then scale up).

Note on the ML Model: The currently-deployed model is **donateacry-cnn-bootstrap-v1** (flagged in code as `BOOTSTRAP_NOT_PRODUCTION`). It serves only to validate the pipeline. It is **not a validated clinical tool**. All documentation, UI labels, and communications must clearly state that cry classifications are provisional and pending future retraining.

Summary of Status: Software integration is complete; hardware integration and dataset collection are next. This document should guide the remaining work and serve as the technical specification and handover for operations and development teams. Each section above should be updated as progress is made and before official launch.
